

CANDIDATE CASE STUDY

Production-Ready Legal Document Processing Pipeline

Backend service case study with an optional frontend task and Docker bonus deliverable

REQUIRED	OPTIONAL	BONUS
Backend API	Frontend UI	Docker setup
Submission timeline Deliverables are expected within 2 days. Candidates should prioritize a clean, reliable backend first; the frontend is optional, and Docker is a bonus.		

1. Objective

You are tasked with building a lightweight, production-ready backend service that extracts structured information from raw, messy text documents simulating legal contracts, validates the extracted data against business rules, and exposes the functionality via a clean API.

2. Technical Constraints NON-NEGOTIABLE

- **Language & Framework:** Python 3.11+ and FastAPI.
- **LLM Orchestration:** Pure Python with the native openai SDK or google-genai SDK. Do not use LangChain or LlamaIndex.
- **Data Validation:** Pydantic v2.
- **Database / Storage:** In-memory or a local SQLite database using SQLAlchemy or SQLModel.
- **Development Environment:** Provide a clean requirements.txt or pyproject.toml.

3. Background

Our firm receives raw, unstructured text summaries of commercial real estate lease agreements. We need an automated pipeline that ingests this text, extracts core contract metadata with reliable data types, calculates a financial risk metric, and stores the results.

4. Core Backend Requirements REQUIRED

4.1 Structured Extraction Pipeline

Create an endpoint that accepts raw text and uses a modern LLM, such as gpt-5-mini or gemini-3.5-flash, to extract structured data using Strict Structured Outputs or JSON Schema enforcement.

POST /api/v1/extract

Field to Extract	Expected Type / Format
Lessor (Landlord)	String
Lessee (Tenant)	String
Commencement Date	ISO 8601 date: YYYY-MM-DD
Expiration Date	ISO 8601 date: YYYY-MM-DD
Monthly Rent	Float or Decimal
Currency	3-letter ISO code, for example USD or AED
Termination Notice Period	Integer number of days

4.2 Deterministic Business Logic & Validation

Do not rely on the LLM to perform business calculations or strict schema validation. Once the raw JSON is returned from the LLM, the backend must perform deterministic validation and derived calculations.

- Validate the payload using a Pydantic v2 schema.
- Calculate Contract Duration (Days) using standard Python datetime math.
- If Expiration Date is before Commencement Date, flag the record as INVALID and return HTTP 422 with validation details.
- If Monthly Rent is negative, flag the record as INVALID and return HTTP 422 with validation details.

4.3 Resilience & Error Handling

- Implement a basic retry mechanism with exponential backoff for the LLM API call. Native Python loops or a library such as tenacity are acceptable.
- Handle LLM API failures, timeouts, and rate limits gracefully.
- Handle potential JSON decoding failures gracefully if the API fails to return valid schema structures.

4.4 API & Storage

- Expose GET /api/v1/contracts to list all successfully processed contracts from the local SQLite or in-memory database.
- Expose GET /api/v1/contracts/{id} to fetch a single contract.
- Only successfully processed contracts should be returned from the list and detail endpoints.

5. Mock Input Data for Testing

Candidates should use the following sample to ensure their solution can handle realistic, messy parsing:

MEMORANDUM OF LEASE

This agreement is entered into this 12th day of May, 2024, by and between Apex Holdings LLC (hereafter the Landlord) and Vertex Tech Solutions Corp (hereafter the Tenant). The property located at Suite 404, Dubai Sports City, is leased for a term starting on June 1st, 2024, and ending exactly two years later on May 31st, 2026. The agreed monthly consideration is 12500.00 AED, payable on the first of each month. Either party may terminate this agreement early by providing at least 90 days written notice to the other party.

6. Deliverables Required WITHIN 2 DAYS

Deliverable	Expectation
Source code	A clean GitHub repository link or a zip file containing the project.
README.md	Setup steps, virtual environment creation, dependency installation, environment variable configuration, and test commands.
API test example	A sample curl command or Postman collection for testing the endpoints.
Architectural note	A short paragraph explaining the prompt structure and system instructions used to support deterministic JSON outputs.
Optional frontend	A separate Next.js or React codebase if the candidate chooses to complete the optional UI task.
Bonus Docker setup	Dockerfiles and optional Docker Compose setup for running the backend and frontend together.

7. Optional Frontend Task **OPTIONAL**

As an optional extension, candidates may build a separate frontend codebase using Next.js or React that connects to the backend API.

Frontend objective

Build a lightweight user interface that allows users to submit raw lease agreement text, trigger the extraction pipeline, view the structured extraction result, and browse previously processed contracts.

7.1 Contract Extraction Page

- Create a page with a large text area where the user can paste raw contract or lease text.
- Add a submit button that calls POST /api/v1/extract.
- Show a loading state while the backend is processing the request.
- Show a success state with the extracted fields: Lessor, Lessee, Commencement Date, Expiration Date, Monthly Rent, Currency, Termination Notice Period, and Contract Duration Days.

7.2 Validation & Error Handling

- Handle backend validation errors gracefully, especially HTTP 422 responses caused by invalid dates or negative rent.
- Display readable error messages for backend unavailable, LLM timeout, invalid response, rate limit, or retry exhaustion scenarios.

7.3 Contracts List Page

Create a page that calls the contracts list endpoint and displays successfully processed contracts in a simple table or card layout.

```
GET /api/v1/contracts
```

- Each record should show: Contract ID, Lessor, Lessee, Commencement Date, Expiration Date, Monthly Rent, Currency, and Contract Duration Days.

7.4 Contract Details Page

Create a details page that fetches and displays the full stored contract record in a clean, readable format.

```
GET /api/v1/contracts/{id}
```

7.5 Frontend Technical Expectations

- Use Next.js or React.
- Keep the frontend in a separate codebase or folder from the backend.
- Use a simple API client layer rather than calling fetch directly from every component.
- Use clear loading, success, empty, and error states.
- Basic styling is sufficient. Tailwind CSS, CSS modules, or plain CSS are acceptable.
- Include a README.md covering setup instructions, required environment variables, backend API base URL configuration, and how to run the frontend locally.

```
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000
```

8. Bonus: Docker Setup **BONUS**

Additional bonus credit will be given if the candidate provides Docker support for the backend and, if completed, the optional frontend.

8.1 Backend Docker Deliverables

- Dockerfile for the FastAPI backend.
- Clear instructions for running the API container.
- The container should expose the FastAPI service, for example on port 8000.
- Environment variable handling should be documented, especially for API keys and runtime configuration.

8.2 Frontend Docker Deliverables

- Dockerfile for the Next.js or React frontend, if the optional frontend task is completed.
- Clear instructions for running the frontend container.
- The container should expose the frontend app, for example on port 3000.

8.3 Full Stack Setup

- Extra credit if the candidate provides a docker-compose.yml file that runs the backend API and frontend app together.
- Include a SQLite or local storage volume if applicable.
- The frontend should communicate with the backend through the configured API base URL when both services are running through Docker Compose.

Important prioritization

Docker is a bonus across the solution. A strong backend implementation should not be penalized for not completing the optional frontend. The optional frontend should be evaluated separately from the required backend.

9. Suggested Submission Checklist

- Backend runs locally with documented setup steps.
- POST /api/v1/extract validates and stores successful records.
- GET /api/v1/contracts and GET /api/v1/contracts/{id} work as expected.
- Pydantic v2 models enforce deterministic validation.
- LLM call uses strict structured outputs or JSON schema enforcement.
- Retries and error handling are implemented for LLM failures.
- README includes curl or Postman examples.
- Optional frontend, if submitted, runs separately and connects to the backend.
- Bonus Docker setup, if submitted, includes clear run instructions.